

LLM Wiki v2 — extending Karpathy's LLM Wiki pattern with lessons from building agentmemory

llm-wiki.md

LLM Wiki v2

A pattern for building personal knowledge bases using LLMs. Extended with lessons from building [agentmemory](#), a persistent memory engine for AI coding agents.

This builds on [Andrej Karpathy's original LLM Wiki idea file](#). Everything in the original still applies. This document adds what we learned running the pattern in production: what breaks at scale, what's missing, and what separates a wiki that stays useful from one that rots.

What the original gets right

The core insight is correct: **stop re-deriving, start compiling**. RAG retrieves and forgets. A wiki accumulates and compounds. The three-layer architecture (raw sources, wiki, schema) works. The operations (ingest, query, lint) cover the basics. If you haven't read the original, start there.

What follows is what we found after building and running this pattern across thousands of sessions.

The missing layer: memory lifecycle

The original treats all wiki content as equally valid forever. In practice, knowledge has a lifecycle. A bug you discovered last week matters more than one from six months ago. A pattern you've seen twelve times is more reliable than one you've seen once. A claim from a newer source should weaken an older one automatically.

Confidence scoring. Every fact in the wiki should carry a confidence score: how many sources support it, how recently it was confirmed, whether anything contradicts it. When the LLM writes "Project X uses Redis for caching," that claim should know it came from two sources, was last confirmed three weeks ago, and sits at confidence 0.85. Confidence decays with time and strengthens with reinforcement. This turns the wiki from a flat collection of equally-weighted claims into a living model where the LLM can say "I'm fairly sure about X but less sure about Y."

Supersession. When new information contradicts or updates an existing claim, the old claim shouldn't just sit there with a note. The new one should explicitly supersede it. Linked, timestamped, old version preserved but marked stale. Version control for knowledge, not just for files.

Forgetting. Not everything should live forever. A wiki that never forgets becomes noisy. Implement a retention curve: facts that were important once but haven't been accessed or reinforced in months should gradually fade. Not deleted, but deprioritized. The LLM equivalent of moving something to a bottom drawer. Ebbinghaus's forgetting curve works well here: retention decays exponentially with time, but each reinforcement (access, confirmation from a new source) resets the curve. Architecture decisions decay slowly. Transient bugs decay fast.

Consolidation tiers. Raw observations aren't the same as established facts. Build a pipeline:

- **Working memory:** recent observations, not yet processed
- **Episodic memory:** session summaries, compressed from raw observations
- **Semantic memory:** cross-session facts, consolidated from episodes
- **Procedural memory:** workflows and patterns, extracted from repeated semantics

Each tier is more compressed, more confident, and longer-lived than the one below it. The LLM promotes information up the tiers as evidence accumulates. This is how you go from "I saw this once" to "this is how things work."

Beyond flat pages: the knowledge graph

The original wiki is pages with wikilinks. That works, but you're leaving structure on the table. What you actually want is a typed knowledge graph layered on top of the pages.

Entity extraction. When the LLM ingests a source, it shouldn't just write prose. It should extract structured entities. People, projects, libraries, concepts, files, decisions. Each entity gets a type, attributes, and relationships to other entities. "React" is a library. "Auth migration" is a project. "Sarah" is a person who owns the auth migration and has opinions about React.

Typed relationships. Not all connections are equal. "uses," "depends on," "contradicts," "caused," "fixed," "supersedes" carry different semantic weight. A link that says "A relates to B" is less useful than "A caused B, confirmed by 3 sources, confidence 0.9."

Graph traversal for queries. When someone asks "what's the impact of upgrading Redis?", the LLM shouldn't just keyword-search. It should start at the Redis node, walk outward through "depends on" and "uses" edges, and find everything downstream. This catches connections that keyword search misses.

The graph doesn't replace the wiki pages. It augments them. Pages are for reading. The graph is for navigation and discovery.

Search that actually scales

The original relies on `index.md`, a single file cataloging every page. This works up to maybe 100-200 pages. Beyond that, the index itself becomes too long for the LLM to read in one pass, and you need real search.

Hybrid search. The best approach combines three streams:

- **BM25** (keyword matching with stemming and synonym expansion)
- **Vector search** (semantic similarity via embeddings)
- **Graph traversal** (entity-aware relationship walking)

Fuse the results with reciprocal rank fusion. Each stream catches things the others miss. BM25 finds exact terms. Vectors find semantic similarity. The graph finds structural connections. Together they beat any single approach.

Keep `index.md` as a human-readable catalog, but don't rely on it as the LLM's primary search mechanism past ~100 pages.

Automation: from manual to event-driven

The biggest practical gap in the original is that everything is manual. You drop a source and tell the LLM to process it. You remember to run lint periodically. You decide when to file an answer back.

In practice, you want hooks. Events that fire automatically:

- **On new source:** auto-ingest, extract entities, update graph, update index
- **On session start:** load relevant context from the wiki based on recent activity
- **On session end:** compress the session into observations, file insights
- **On query:** check if the answer is worth filing back (quality score > threshold)
- **On memory write:** check for contradictions with existing knowledge, trigger supersession
- **On schedule:** periodic lint, consolidation, retention decay

The human should still be in the loop for curation and direction. But the bookkeeping, the part that makes people abandon wikis, should be fully automated.

Quality and self-correction

Not all LLM-generated content is good. Without quality controls, the wiki accumulates noise.

Score everything. Every piece of content the LLM writes should get a quality score. Is it well-structured? Does it cite sources? Is it consistent with the rest of the wiki? You can have the LLM self-evaluate, or use a second pass with a different prompt. Content below a threshold gets flagged for review or rewritten.

Self-healing. The lint operation from the original should be more than a suggestion. It should automatically fix what it can. Orphan pages get linked or flagged. Stale claims get marked. Broken cross-references get repaired. The wiki should tend toward health on its own, not only when you remember to ask.

Contradiction resolution. The original mentions flagging contradictions. That's step one. Step two is resolving them. The LLM should propose which claim is more likely correct based on source recency, source authority, and the number of supporting observations. The human can override, but the default behavior should usually be right.

Multi-agent and collaboration

The original is single-user, single-agent. Many real use cases involve multiple agents or multiple people contributing to the same knowledge base.

Mesh sync. If multiple agents are working in parallel (different coding sessions, different research threads), their observations need to merge into a shared wiki. Last-write-wins works for most cases. For conflicts, timestamp-based resolution with manual override.

Shared vs. private. Some knowledge is personal (my preferences, my workflow). Some is shared (project architecture, team decisions). The wiki needs scoping. Private observations that roll up into shared knowledge when promoted.

Work coordination. When multiple agents work on the same knowledge base, they need lightweight coordination. Who's working on what. What's blocked. What's done. Not a full task management system, just enough to prevent duplicate work and track progress.

Privacy and governance

The original doesn't mention this, but it matters. Sources often contain sensitive information: API keys, credentials, private conversations, PII.

Filter on ingest. Before anything hits the wiki, strip sensitive data. API keys, tokens, passwords, anything marked private. This should be automatic, not something you remember to do.

Audit trail. Every operation on the wiki (ingest, edit, delete, query) should be logged with a timestamp, what changed, and why. This is your accountability layer. When something looks wrong in the wiki, the audit trail tells you how it got there.

Bulk operations with governance. As the wiki grows, you'll want to bulk-delete stale content, export subsets, or merge duplicate entities. These operations should be audited and reversible.

Crystallization: compounding from exploration

The original mentions that "good answers can be filed back into the wiki as new pages." This can be taken further.

Crystallization is the process of taking a completed chain of work (a research thread, a debugging session, an analysis) and automatically distilling it into a structured digest. What was the question? What did we find? What files/entities were involved? What lessons emerged? This digest becomes a first-class wiki page, and the lessons get extracted as standalone facts that strengthen the knowledge base.

Your explorations are a source, just like an article or a paper. The wiki should treat them that way. Ingest the results, update the graph, strengthen or challenge existing claims.

Output formats beyond markdown

The original mentions Marp for slide decks and matplotlib for charts. The wiki's output shouldn't be limited to markdown pages. Depending on the query, the right output might be:

- A comparison table
- A timeline visualization
- A dependency graph
- A slide deck for presenting findings
- A structured data export (JSON, CSV) for further analysis
- A brief for someone else on your team

The wiki is the knowledge store. The output format depends on the audience and the question.

The schema is the real product

The original implies this but it's worth being direct: **the schema document (CLAUDE.md, AGENTS.md) is the most important file in the system.** It's what turns a generic LLM into a disciplined knowledge worker. It encodes:

- What types of entities and relationships exist in your domain
- How to ingest different kinds of sources
- When to create a new page vs. update an existing one
- What quality standards to apply
- How to handle contradictions
- What the consolidation schedule looks like
- What's private vs. shared

You and the LLM co-evolve this document over time. The first version will be rough. After a few dozen sources and a few lint passes, you'll have a schema that reflects how your domain actually works. That schema is transferable. Share it with someone else working on a similar domain and they get a running start.

Implementation spectrum

All of this is modular. You don't need everything on day one.

Minimal viable wiki: raw sources + wiki pages + index.md + a schema that describes ingest/query/lint workflows. This is roughly what the original describes. It works. Start here.

Add lifecycle: confidence scoring, supersession, basic retention decay. This prevents the wiki from becoming a junk drawer.

Add structure: entity extraction, typed relationships, knowledge graph. This makes queries better and surfaces connections you'd miss with flat pages.

Add automation: hooks for auto-ingest, auto-lint, context injection. This is where the maintenance burden drops to near zero.

Add scale: hybrid search, consolidation tiers, quality scoring. This is what you need when the wiki grows past a few hundred pages.

Add collaboration: mesh sync, shared/private scoping, work coordination. This is for teams or multi-agent setups.

Pick your entry point based on your needs. The pattern works at every level.

Why this matters

Karpathy's original insight stands: the bottleneck is bookkeeping, and LLMs eliminate that bottleneck. What we've added is the machinery that keeps the wiki healthy as it scales. Lifecycle management so knowledge doesn't rot. Structure so connections aren't lost. Automation so humans stay focused on thinking rather than filing. Quality controls so the wiki earns trust over time.

The Memex is finally buildable. Not because we have better documents or better search, but because we have librarians that actually do the work.

This document extends [Andrej Karpathy's LLM Wiki](#) with patterns proven in [agentmemory](#), a persistent memory engine for AI agents built on [iii-engine](#). The original idea file is the foundation; this adds what we learned building the engine.



luancaarvalho commented 3 weeks ago

Hello!

Great approach, I had the same impression when reading Karpathy's article and also some videos about his approach. The only point I didn't understand was the following: based on your experience, the scaling problem that Karpathy mentioned in his article boils down to migrating from an index file that works for 200 documents to a hybrid search across those documents. So, we continue to generate their graph in Obsidian and this, along with BM25 and vector search, will solve the scaling problem, correct?

webmaven commented 3 weeks ago

How well does the ingestion scale to larger source documents, like a book (or a manuscript)?

rohitg00 commented 3 weeks ago

Author

Hello! Great approach, I had the same impression when reading Karpathy's article and also some videos about his approach. The only point I didn't understand was the following: based on your experience, the scaling problem that Karpathy mentioned in his article boils down to migrating from an index file that works for 200 documents to a hybrid search across those documents. So, we continue to generate their graph in Obsidian and this, along with BM25 and vector search, will solve the scaling problem, correct?

Good question [@luancaarvalho](#), Short answer: mostly yes, but with a caveat.

A flat index file breaks around 200-500 documents because you're doing brute-force matching. BM25 + vector search handles the retrieval scaling. BM25 catches exact terms, vectors catch semantic similarity, and RRF fusion ranks the combined results.

But the graph layer matters too. Obsidian-style graphs capture relationships between documents that neither BM25 nor vector search surface on their own. In agentmemory we run all three (BM25 + vector + knowledge graph) and fuse with RRF, that's how we hit 95.2% on LongMemEval-S.

So it's not just "replace index with hybrid search." It's **index → hybrid retrieval + graph traversal** working together. The graph tells you what's connected. The hybrid search tells you what's relevant right now.

rohitg00 commented 3 weeks ago

Author

How well does the ingestion scale to larger source documents, like a book (or a manuscript)?

[@webmaven](#)

Haven't stress-tested book-length ingestion yet, current design is optimized for agent observations (short-to-medium text chunks from coding sessions, conversations, tool outputs).

For a book or manuscript, the bottleneck would be chunking strategy and how observations are created from the source material. You'd likely want to pre-process into meaningful segments (chapters, sections, key passages) rather than feeding raw text. The retrieval layer should handle it fine. BM25 and vector search scale well with document count. The knowledge graph extraction would be the expensive part at that volume.

Interested in the use case, are you thinking about an agent that builds memory from reading a manuscript, or more like a research assistant that accumulates knowledge across multiple books?

ttaskippythemagnificent-coder commented 3 weeks ago

YOINK! Gracias,

Item 3: Rohit's LLM Wiki Knowledge System Gist

Verdict: STEAL THIS. High-value blueprint.

This is a production-focused architecture doc based on Karpathy's LLM Wiki pattern — updated yesterday. Not an awesome-list. A working blueprint covering:

- Memory architecture with confidence scoring
- Hybrid search (BM25 + vectors + graph traversal)
- Knowledge graph structuring and quality controls
- Multi-agent sync patterns
- Schema-driven knowledge work

This maps directly onto our stack evolution:

- Semantic search (Phase 2, done) → hybrid search is our Phase 3
- Knowledge graph (582 nodes) → confidence scoring and quality controls are exactly what we need for enrichment
- Multi-agent patterns → we already have 11 crew officers that could benefit from shared knowledge sync
- BM25 + vector + graph — the triple retrieval pattern is the upgrade path for our semantic search

Action: Pull the gist content, extract the architectural patterns, cross-reference against our knowledge graph roadmap (Project Lick). The hybrid search layer and confidence scoring are the two pieces we should incorporate into Phase 3 planning.

We already have 60% of what this blueprint describes. Semantic search, knowledge graph, ingest pipeline, lint, hooks — all built. The missing 40% is the intelligence layer:

1. Phase 3A — Confidence scores on facts (the unlock for everything else)
2. Phase 3B — Wire graph into search (we have both pieces, just not connected)
3. Phase 3C — Auto-crystallize sessions into knowledge (compounding loop)
4. Phase 3D — Contradiction detection on write (quality control)
5. Phase 3E — Multi-agent knowledge scoping (deferred until crew needs it)

3A + 3B together in one golden window session is the sovereignty leap — ~8-10 hours, turns our search from "find stuff" into "associative intelligence with memory lifecycle." That's the Simmerer submind's fuel.

 **gnusupport** commented 3 weeks ago

[@ttaskippythemagnificent-coder](#)

- "Confidence scoring" is never defined — float? enum? who computes it? how does it update?
- "Auto-crystallize sessions into knowledge" is pure magic — no extraction algorithm, no dedup, no trigger condition
- 582 nodes is tiny — that's not a knowledge graph, that's a Tuesday afternoon
- Hybrid search has no fusion strategy — BM25 + vectors + graph just means three slow things bolted together
- No latency targets — 100ms? 10 seconds? who knows?
- No accuracy metrics — NDCG? MRR? nothing
- "8-10 hours for 3A+3B" is delusional — that's a notebook hack, not production
- "We already have 60%" assumes components compose — they won't; integration is the work
- Contradiction detection on every write is AI-complete — they're not ready for that complexity

- No access control — any agent can write anything
- No versioning — can't roll back a bad crystallization
- No provenance — which agent wrote this fact? from what source?
- No consistency model — ACID? eventual? multi-agent sync needs this
- No backup or recovery strategy — corruption = game over
- No evaluation framework — how do you know if it worked?
- LLMs are treated as reliable — they'll silently corrupt the graph and you'll never know
- No human-in-the-loop for LLM failures
- No fallback when LLM API is down
- No human-readable addresses — can't cite a fact in an email or doc
- No back-links — "what points to this node?" is missing
- No time stamps — can't audit when a fact changed
- No signature or authentication — can't trust who wrote what
- No external document linking (XDoc) — Slack, email, PDFs stay outside
- The timeline assumes no bugs, no testing, no security, no rollback
- It's a product vision dressed as an architecture document

Great direction, terrible blueprint. Don't build from this. Steal the *ideas*, not the *plan*.

Learn:

TECHNOLOGY TEMPLATE PROJECT OHS Framework :

<https://www.doungelbart.org/content/view/110/460/>



tigerlaibao commented 3 weeks ago

"The Memex is finally buildable" — this line in your Gist resonated deeply with me.

You hit the nail on the head: the biggest bottleneck isn't the data, it's the "bookkeeping tax." Most systems rot because they require humans to act like librarians.

I've spent the last few months building Memex, an open-source tool that attempts to productize these exact patterns. I wanted to move beyond a "hacky collection of scripts" into a polished, cross-platform experience.

How I'm approaching this:

Automation of the "Chore": Memex uses AI agents to incrementally "compile" daily captures into a structured P.A.R.A. library. The goal is: Capture freely, organized automatically.

The Engine (dart_agent_core): To power these agentic workflows in a Flutter app, I built a dedicated framework: dart_agent_core. It's a stateful AI agent framework for Dart that supports tool use, planning, and sub-agent delegation.

Product over Scripts: I chose Flutter because a knowledge base needs a high-performance, beautiful UI to stay sustainable. It's about keeping the "system" out of the way of the "mood."

The project is in its early stages, but the core loop of "raw ingest -> agentic compilation -> structured wiki" is live. I'd love for you to take a look!

App: <https://github.com/memex-lab/memex>

Framework: https://github.com/memex-lab/dart_agent_core



stevesolun commented 2 weeks ago

It is definitely a great approach but much powerful with the knowledge graph full of skills and agents I have built:

I got tired of manually figuring out which skill or agent to use inside Claude Code.

So I built ctx.

ctx watches what you're developing, walks a knowledge graph of 1,450+ skills and 427 agents, and recommends the right ones in real time.

You stay in control:

load what you want

unload what you do not need

keep the context sharp

Under the hood, it uses a Karpathy-style LLM wiki, plus persistent memory that gets smarter across sessions.

<https://github.com/stevesolun/ctx>



ChristopherA commented 2 weeks ago

I'm interested in this category, but less inferred graphs for knowledge but instead explicitly authored named edges (initiated by people but facilitated by agents).

<https://gist.github.com/ChristopherA/151aefa6a6bde1ce4fa6b1182656cebe>

Summary: A practical guide to turning a folder of markdown files into a real knowledge graph using just two conventions: `[[wikilinks]]` for connections and named edges like `derived_from::[[Source]]` for what the connection means. No database, no special tooling — just text files and discipline. Worth reading if you keep notes in Obsidian-style tools, build AI agents that traverse structured knowledge, or care about how collaborators can share a graph without surrendering their own vocabulary to someone else's schema.



silentrob commented 5 days ago

I have a fork of quicky-wiki with entity, graph, and bm25 + vector support and came to the same conclusions. <https://github.com/silentrob/quicky-wiki>



wking1986 commented 3 days ago

Great summary! The design philosophy behind LLM Wiki v2 is quite similar to that of the Gbrain
<https://github.com/garrytan/gbrain>